Soham Nayak

DSA Unit 1 - Questions and Answers

DSA Question and Answers

Section A: Conceptual Understanding

Q2. What is Information? How is it different from data? Provide one example to illustrate the transformation.

Answer:

Information is processed, structured, and meaningful output derived from raw data. While data is a collection of unorganized facts, information provides context and supports decision-making.

Difference:

Data: Raw, unprocessed facts (e.g., numbers, text, images).

Information: Data that has been analyzed and interpreted to make it useful.

Example:

Data: A set of temperature readings over a week – [31, 33, 34, 36, 34, 35, 37]

Information: "The average temperature this week was 34.3°C, indicating a rising heat trend."

Here, data was converted into actionable insight through averaging, helping perhaps a weather department decide whether to issue a heatwave warning.

Section B: Tabular Analysis

Q5. Recreate the table comparing Data vs Information as shown in class. Add one new example of your own for each column.

Answer:

This table captures the essential distinction—data is the input, information is the actionable output.

Section C: Application-Based Questions

Q6. For each of the following domains, identify: one piece of raw data, how it is processed, and what information it produces.

Answer:

Each scenario shows transformation from raw data to domain-specific insights.

Section D: Visualization-Based

Q8. Using the example of the retail manager during holiday season (as shown in slides):

Write how each characteristic (Relevance, Completeness, Timeliness, Verifiability) was fulfilled. Add one more characteristic you think is important and justify.

Answer:

Relevance: Sales data from previous holidays was shown; this directly supports seasonal inventory decisions.

Completeness: Data included customer preferences, regional demand, and product categories—ensuring well-rounded analysis.

Timeliness: Data was updated to include last week's figures—not outdated trends—helping forecast current needs.

Verifiability: The source system logs and timestamps validated the origin and trustworthiness of the data.

Additional Characteristic – Accuracy:

Even if data is timely and complete, inaccurate data leads to poor decisions. For example, wrongly recorded sales can lead to stockouts or overstocking. Thus, accuracy is critical for trust and utility in decision-making.

Section E: Organizational Significance

Q10. Describe how data is an asset for modern organizations. Mention at least three use-cases where data drives outcomes (from slides).

Answer:

Data is often called the "new oil" because it fuels decision-making, automation, and innovation. When collected, cleaned, and processed well, data adds tangible value to any organization.

Three Use-Cases:

Personalized Marketing: By analyzing browsing and purchase history, e-commerce platforms like Amazon tailor product suggestions.

Operational Efficiency: Manufacturing units use machine sensor data to predict equipment failures and optimize workflows (predictive maintenance).

Strategic Planning: Retail chains analyze regional sales data to decide new store locations and inventory stocking strategies.

In all these cases, data transforms from a passive asset to a strategic tool for gaining competitive advantage.

Q4. (Bloom's Level: Apply → Analyze)

Question:

Think of two ways of arranging items:

• Lining up books on a shelf (one after another)

• Creating a family tree (branching from one to many)

Which one reflects linear arrangement? Which one is non-linear? Where would you use each in real life?

Answer:

Linear Arrangement is like lining up books on a shelf. Items are stored one after another in a single sequence.

âž¤ Real-life example: Attendance list, where each student's name is written in a row one after the other.

Non-linear Arrangement is like a family tree. Each item can branch out into multiple items, creating a hierarchy or network.

âž¤ Real-life example: File organization in a computer folder system or organization chart in a company.

When to use:

Use linear structures when order and sequence matter (e.g., processing customer complaints in order).

Use non-linear structures when representing relationships or hierarchies (e.g., a company's employee hierarchy).

Q5. (Bloom's Level: Understand → Evaluate)

Question:

Why is it useful to classify data structures as static/dynamic, linear/non-linear, and so on? If you are a software planner for a supermarket billing system, how would knowing these help in choosing the right way to organize the data?

Answer:

Classifying data structures helps developers make better design choices based on:

Flexibility needs (static vs. dynamic)

Access patterns (linear vs. non-linear)

Memory management (pre-allocated vs. resizeable)

Speed and efficiency of operations (search, insert, delete)

Example in Supermarket Billing:

Product list may use a dynamic linear structure (like a growing list) to add/remove items during checkout.

Customer queue can use a linear structure (like a queue or list) as each person waits in order.

Offer lookup table might use a non-linear structure (like a tree or hash) for faster access.

Without classification, you might choose an inefficient structure that increases time or memory usage, affecting performance.

Q8. (Bloom's Level: Understand → Analyze)

Question:

Explain the importance of encapsulation using a real-life analogy. For example, a vending machine hides how it works internally and only shows buttons to press. Why is such hiding useful in digital systems?

Answer:

Encapsulation means hiding internal details and exposing only necessary functionality.

Real-life analogy: A vending machine shows only buttons and displays. You don't need to know how the gears or coin mechanisms work.

Similarly, a mobile app might expose "Add to Cart" but hide the database logic underneath.

Why is this useful?

Prevents accidental interference with internal operations.

Makes systems easier to use and update.

Allows developers to change internals without affecting users.

Improves security, modularity, and maintenance.

It enables building reliable and user-friendly systems where the complexity is managed behind the scenes.

Q9. (Bloom's Level: Create)

Question:

Choose one system around you (like a library, classroom, or attendance sheet).

• What kind of data is stored?

• Suggest a simple way to structure it: fixed or flexible, arranged in a list or a group, same type or mixed?

• How would you access or update this data?

Answer:

System Chosen: Library Book Management System

Data Stored:

Book Title (Text), Author (Text), ISBN (Number), Genre (Text), Availability (Yes/No)

Structure Suggestion:

Use a flexible structure (to allow adding/removing books).

Grouped records (each record stores all details about one book).

Heterogeneous data (mix of text, numbers, and logical values).

Access/Update Strategy:

Use a search feature by title or ISBN.

If found, update availability status (borrowed/returned).

Add or remove records for new or removed books.

This design allows librarians to track and manage the collection easily, and helps users quickly search for available books.

Q10. (Analyzing)

Identify and fix errors in the following code:

```
struct Student {
   int roll;
   char name[30];
};
```

```
int main() {
   struct Student s1;
   s1.name = "Ravi"; // Fix this
   return 0;
}
```

Answer:

In C, you cannot assign a string directly to a character array using = after declaration.

You should use strcpy() from the string.h library.

Corrected Code:

```
#include <stdio.h>
#include <string.h> // Needed for strcpy

struct Student {
   int roll;
   char name[30];
};

int main() {
   struct Student s1;
   strcpy(s1.name, "Ravi"); // Correct usage
   return 0;
}
```

Why it's wrong:

s1.name is an array, and arrays cannot be assigned directly with = in C. They must be copied character by character (as done by strcpy()).

Q11. (Analyzing)

Consider a structure that holds employee salary details. Why might structure padding affect memory usage?

Answer:

Structure padding is the addition of empty memory bytes between members to align data properly according to the CPU architecture.

Example:

```
struct Employee {
   char grade;    // 1 byte
   float salary;  // 4 bytes (aligned at 4 bytes)
};
```

Memory layout:

grade takes 1 byte

Then 3 padding bytes are added to align salary

salary takes 4 bytes

Total = 8 bytes, not 5

Why this matters:

Wastes memory when many structure variables are used.

Becomes important in systems with limited memory or large datasets.

Optimization Tip:

Reorder structure members from largest to smallest types to minimize padding.

Q12. (Evaluating)

Given two structure variables with identical fields, can you use == to compare them? Justify your answer with a code example.

Answer:

No, C does not allow direct comparison of structures using ==.

Example:

```
struct Point {
   int x, y;
};

struct Point p1 = {2, 3};
struct Point p2 = {2, 3};

if (p1 == p2) // â• Œ Error: Invalid operands
   printf("Equal");
```

Correct way:

```
if (p1.x == p2.x && p1.y == p2.y)
   printf("Equal");
```

Why not allowed:

C does not define how structure comparison should work; memory layout may vary due to padding.

Alternative:

Use a custom function to compare fields manually.

Q15. (Creating)

Create a nested structure for Student which includes a nested structure Date for storing date of birth. Write a function to input and print this data.

Answer:

```
#include <stdio.h>

struct Date {
   int day, month, year;
};

struct Student {
   char name[30];
   int roll;
   struct Date dob; // Nested structure
};

void inputStudent(struct Student *s) {
   printf("Enter name: ");
   scanf("%s", s->name);
   printf("Enter roll number: ");
   scanf("%d", &s->roll);
   printf("Enter date of birth (dd mm yyyy): ");
   scanf("%d %d %d", &s->dob.day, &s->dob.month, &s->dob.year);
}

void printStudent(struct Student s) {
   printf("Name: %s\n", s.name);
   printf("Roll No: %d\n", s.roll);
   printf("DOB: %02d-%02d-%04d\n", s.dob.day, s.dob.month, s.dob.year);
}

int main() {
   struct Student s;
   inputStudent(&s);
   printStudent(s);
   return 0;
}
```

Explanation:

Nested structures enable clean grouping.

Helps in real-world modeling like DOB, address, etc.

Q17. (Creating)

Define a struct Product with a const int productID. Try to modify it inside a function and observe the compiler's behavior. Comment on the outcome.

Answer:

```
#include <stdio.h>

struct Product {
   const int productID;
   float price;
};

void updateProduct(struct Product *p) {
   // p->productID = 500; //  Compiler Error: assignment of read-only member
   p->price = 99.99;
}

int main() {
   struct Product p = {101, 50.0};
   updateProduct(&p);
   printf("Product ID: %d, Price: %.2f\n", p.productID, p.price);
   return 0;
}
```

Observation:

The line p->productID = 500; will throw an error:

"assignment of read-only member 'productID'"

Conclusion:

Declaring a member const makes it immutable after initialization.

Useful for fields like IDs, registration numbers, or serials that must not change

Q4. (Analyze)

Analyze why only the last assigned member in a union retains its value. Support with a simple code example.

Answer:

A union uses a shared memory space for all its members. This means all members occupy the same memory location. When you assign a value to one member, it overwrites the memory previously used by any other member.

Key Point:

Only one member's value can be valid at a time.

Assigning to a different member will corrupt or replace the existing value.

Example:

```
#include <stdio.h>

union Data {
   int i;
   float f;
};

int main() {
   union Data d;
   d.i = 10;
   printf("After assigning int: d.i = %d\n", d.i);

d.f = 3.14;
   printf("After assigning float: d.f = %f\n", d.f);
   printf("Now d.i (invalid): %d\n", d.i); // Value corrupted
   return 0;
}
```

Output:

```
After assigning int: d.i = 10
After assigning float: d.f = 3.140000
Now d.i (invalid): <garbage>
```

Conclusion:

When d.f is written, it overwrites the bits originally held by d.i.

Accessing an overwritten member results in undefined or garbage values.

Q5. (Evaluate – Application-based)

Compare unions and structures in terms of memory usage and purpose. In which scenarios would you prefer unions?

Answer: Comparison:

Use Case for Union:

Unions are ideal when a variable can hold different types at different times, not simultaneously.

When to Use Unions:

Embedded systems (e.g., sensor reading may be temperature OR pressure).

Network protocol packets (e.g., different message types).

Space-constrained environments where saving bytes is critical.

Example:

```
union Packet {
   int id;
   float temp;
};
```

Caution: Using unions incorrectly (e.g., reading an unassigned member) can lead to undefined behavior.

Q6. (Create – Code-based)

Design a union-based structure to represent a protocol message with two variants: a login message (string) and a logout code (integer). Use a tag and write a function to print the message based on the tag.

Answer:

```
#include <stdio.h>
#include <string.h>

#define LOGIN 1
#define LOGOUT 2

struct Message {
   int tag;  // To indicate message type
   union {
      char username[50];  // Login message
      int logoutCode;     // Logout message
   } data;
};

void printMessage(struct Message msg) {
   if (msg.tag == LOGIN) {
      printf("Login Message: User = %s\n", msg.data.username);
   } else if (msg.tag == LOGOUT) {
      printf("Logout Message: Code = %d\n", msg.data.logoutCode);
   } else {
      printf("Unknown message type\n");
```

```
  }
}

int main() {
  struct Message m1, m2;

// Login message
  m1.tag = LOGIN;
  strcpy(m1.data.username, "Alice");
  printMessage(m1);

// Logout message
  m2.tag = LOGOUT;
  m2.data.logoutCode = 200;
  printMessage(m2);

return 0;
}
â€‹â€‹â€‹â€‹â€‹â€‹
```

Output:

```
Login Message: User = Alice
Logout Message: Code = 200
```

Concept Applied:

Union allows memory-efficient storage of alternative data types.

The tag helps identify which type is currently stored.

This simulates a variant or tagged union, common in protocol/message systems.

Q7. (Analyze)

What is wrong with this pointer declaration? Correct and explain.

```
int* p1, p2;
```

Answer:

The declaration is misleading. It creates:

p1 → a pointer to int

p2 → a regular int (not a pointer)

Misconception: People assume both p1 and p2 are pointers due to the placement of *, but only p1 is a pointer.

Correct Way:

int *p1, *p2; // both are pointers to int

Preferred Style:

Always place * next to variable name, not type:

int *p1;   // clear that p1 is a pointer

This avoids confusion when declaring multiple variables.

Q9. (Evaluate)

Evaluate and explain the behavior of this program. Is it valid to return a pointer this way?

```
int* getVal() {
   int x = 5;
   return &x;
}
```

Answer:

This code is invalid. It returns the address of a local variable, x.

Once the function exits, the memory for x is deallocated (stack memory).

So, the returned pointer becomes a dangling pointer → leads to undefined behavior.

Consequences:

May crash the program.

May print garbage or cause segmentation fault.

Safe Alternative:

Use static variable or dynamic memory:

```
int* getVal() {
   static int x = 5;
   return &x;
}
```

Or use malloc (must free later):

```
int* getVal() {
   int* ptr = malloc(sizeof(int));
   *ptr = 5;
```

```
    return ptr;
}
```

Q11. (Create)

Create a program that accepts 5 integers from the user, stores them in an array, and prints the square of each number using pointers only (no indexing).

Answer:

```
#include <stdio.h>

int main() {
    int arr[5];
    int *ptr = arr;

printf("Enter 5 integers:\n");
    for (int i = 0; i < 5; i++) {
        scanf("%d", ptr + i);
    }

printf("Squares of the entered numbers:\n");
    for (int i = 0; i < 5; i++) {
        printf("%d squared = %d\n", *(ptr + i), (*(ptr + i)) * (*(ptr + i)));
    }

return 0;
}
```

Key Concepts:

Uses pointer arithmetic *(ptr + i) instead of arr[i].

Demonstrates input and output fully via pointers.

Q14. (Create)

Create a pointer-based implementation of a simple temperature conversion system:

Accepts temperature value (float) via pointer

Converts it from Celsius to Fahrenheit and stores it back using dereferencing

Answer:

```
#include <stdio.h>

void convertCtoF(float *tempC) {
    *tempC = (*tempC * 9 / 5) + 32;
}
```

```
int main() {
   float temp;
   printf("Enter temperature in Celsius: ");
   scanf("%f", &temp);

convertCtoF(&temp);

printf("Temperature in Fahrenheit: %.2f\n", temp);
   return 0;
}
```

Key Concepts:

Uses pointer to modify value in-place.

Converts Celsius to Fahrenheit using formula:

$F = (C \times 9/5) + 32$

Q16. (Analyze)

Code Analysis:

```
int *p = malloc(5 * sizeof(int));
p = malloc(10 * sizeof(int));
```

Analysis:

The first allocation (malloc(5 * sizeof(int))) is lost when p is reassigned.

This results in a memory leak — 20 bytes (if int = 4 bytes) are lost.

Correction:

```
int *p = malloc(5 * sizeof(int));
if (p == NULL) { /* handle error */ }

int *temp = realloc(p, 10 * sizeof(int));
if (temp != NULL) {
   p = temp;  // safe to reassign
} else {
   // realloc failed; p still points to original memory
}
```

Or:

```
free(p);  // explicitly free before reassigning
p = malloc(10 * sizeof(int));
```

Q17. (Analyze)

Compare: Null vs Dangling vs Wild Pointer

Conclusion:

Always initialize pointers, avoid using after free(), and check before dereferencing.

Q20. (Evaluate)

Justify the need for memory checks after dynamic allocation:

```
int *arr = malloc(100 * sizeof(int));
if (arr == NULL) {
   printf("Memory allocation failed!\n");
   exit(1);
}
```

Why Needed:

Ensures the program doesn't crash unexpectedly.

Especially important in:

Embedded systems

Large-scale applications

Low-memory environments

Best Practices:

Always check return value of malloc, calloc, realloc.

Handle NULL conditions gracefully with error messages or fallback logic.

Q21. (Create)

Generic Memory-Copy Function using void * and memcpy:

```
#include <stdio.h>
#include <string.h>

void copy_memory(void *dest, const void *src, size_t size) {
   memcpy(dest, src, size);
}

int main() {
   int a = 10, b = 0;
   copy_memory(&b, &a, sizeof(int));
   printf("Copied int value: %d\n", b);
```

```
float f1 = 3.14, f2 = 0;
   copy_memory(&f2, &f1, sizeof(float));
   printf("Copied float value: %.2f\n", f2);
   return 0;
}
```

Highlights:

Works with any data type.

Relies on void * for generic typing and memcpy() for block copying.

Q23. (Create)

Allocate and Populate 2D Matrix using Double Pointers:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void fillMatrix(int **matrix, int rows, int cols) {
   for (int i = 0; i < rows; i++)
      for (int j = 0; j < cols; j++)
         matrix[i][j] = rand() % 100;
}

void printMatrix(int **matrix, int rows, int cols) {
   for (int i = 0; i < rows; i++) {
      for (int j = 0; j < cols; j++)
         printf("%3d ", matrix[i][j]);
      printf("\n");
   }
}

int main() {
   int rows = 3, cols = 4;
   int **matrix = malloc(rows * sizeof(int*));
   for (int i = 0; i < rows; i++)
      matrix[i] = malloc(cols * sizeof(int));

srand(time(NULL));
   fillMatrix(matrix, rows, cols);
   printMatrix(matrix, rows, cols);

for (int i = 0; i < rows; i++) free(matrix[i]);
   free(matrix);
```

return 0;
}

Key Concepts:

Uses int ** for 2D dynamic allocation.

Populates matrix with random values.

Frees memory properly to avoid leaks.

Q4. (Analyze)

Q: How does column-major order differ from row-major order in memory layout? Explain with a use-case where column-major might be preferred.

Answer:

In row-major order (used by C), elements are stored row-by-row. For A[3][4], the order is:

A[0][0], A[0][1], A[0][2], A[0][3], A[1][0], ..., A[2][3].

In column-major order (used by Fortran, MATLAB), elements are stored column-by-column:

A[0][0], A[1][0], A[2][0], A[0][1], ..., A[2][3].

Difference:

Row-major optimizes for row-wise traversal.

Column-major optimizes for column-wise access.

Use-case for column-major:

In scientific computing or matrix algebra, column-wise computations (e.g., LU decomposition, eigenvalue problems) are common, and languages like MATLAB use column-major for better cache performance in such tasks.

Q7. (Apply)

Q: For a 3D array int A[2][3][4] with base address 1000 and element size 4 bytes, compute the memory address of A[1][2][3].

Answer:

Use the formula for row-major 3D array:

Address = Base + ((i × y × z) + (j × z) + k) × element size

Given:

i = 1, j = 2, k = 3

y = 3, z = 4

Base = 1000, size = 4 bytes

{Offset} = (1 × 3 × 4) + (2 × 4) + 3 = 12 + 8 + 3 = 23

{Address} = 1000 + 23 × 4 = 1000 + 92 = {1092}

Q15. (Apply)

Q: Consider a recursive function to compute factorial. What is its space complexity and why?

Answer:

```
int factorial(int n) {
   if (n == 0) return 1;
   return n * factorial(n - 1);
}
```

Each recursive call adds a new frame to the call stack, until n == 0.

If n = 5, 6 function calls are made (5 down to 0).

Hence, space used = proportional to n.

Space Complexity = O(n)

Because the call stack grows linearly with n.

Q16. (Evaluate)

Q: In a memory-constrained embedded system, which factors would you consider while selecting an algorithm? Justify your response with an example.

Answer:

In such systems, the following factors are critical:

Space Complexity: Avoid high memory usage.

Predictability: Deterministic timing is crucial.

Low CPU Usage: Save power in battery-operated devices.

Avoid Recursion: Due to stack size limitations.

Example:

For sorting in a smart sensor with limited RAM:

Avoid Merge Sort (extra space), Quick Sort (stack depth).

Prefer Selection Sort or Insertion Sort: O(1) space, simple code.

So, we choose algorithms based on compact memory footprint and predictable behavior.

Q5. (Apply)

Q: Convert the array access expression A[i][j] into equivalent pointer arithmetic notation.

Answer:

In row-major order, for a 2D array A[R][C],

A[i][j] ≡ *(*(A + i) + j)

Alternatively, treating A as a flat pointer:

*(&A[0][0] + (i * C + j))

Both notations show how A[i][j] is accessed by offset from base, illustrating how pointer arithmetic maps to array indexing.

Q4. (Analyze)

Question: Analyze the impact of removing the “uniform access time” assumption in the RAM model. What aspects of analysis would be affected?

Answer:

Removing the uniform access time assumption introduces variability in operation cost due to factors like cache hierarchy, memory latency, and instruction pipelining. This affects time complexity estimation, as basic operations (e.g., addition, memory access) no longer have constant cost. An algorithm that appears optimal in the RAM model may perform worse on real hardware if it leads to frequent cache misses or unaligned memory access. Thus, time predictions become hardware-dependent, reducing the generality of algorithm analysis.

Q6. (Create)

Question: Design a new scenario where RAM model assumptions could mislead performance predictions. Explain your reasoning.

Answer:

Consider a matrix multiplication algorithm optimized for row-major access. Under the RAM model, iterating in row-major or column-major order has the same cost. However, in

practice, accessing in column-major order on a row-major layout causes cache thrashing and increased memory latency. RAM model underestimates this penalty. This scenario shows how cache-unaware code may appear efficient in theory but perform poorly in real systems, misguiding developers.

Q10. (Analyze)

Question: Compare the analysis of the same algorithm using both the RAM model and practical execution profiling.

Answer:

Using the RAM model, we count each operation (assignment, comparison) as one unit and derive an abstract step count. In practical profiling, actual execution time is measured, which includes memory hierarchy effects, CPU pipeline stalls, and instruction caching. While the RAM model gives an upper-bound estimate, profiling shows real-world behavior. Discrepancies occur especially when hardware-level optimizations or bottlenecks influence runtime. For example, loop unrolling or SIMD may accelerate operations that the RAM model treats uniformly.

Q16. (Evaluate)

Question: Evaluate the risks of using void * in large-scale software development. Suggest best practices to mitigate them.

Answer:

Using void * sacrifices type safety, making code error-prone and hard to debug. Type mismatches can go undetected until runtime, leading to undefined behavior. In large systems, this risk compounds as codebases grow and multiple developers interact with shared memory. Best practices include wrapping void * usage in well-documented, encapsulated APIs, validating type at runtime (e.g., tagging), and minimizing its use in favor of typed pointers or generic programming (like macros or templates in C++).

Q32. (Apply)

Question: Analyze the code below and write the total step count as a mathematical expression:

```
for (int i = 0; i < n; i++) {

for (int j = 0; j < n; j++) {

C[i][j] = A[i][j] + B[i][j];

}

}
```

Assume each addition and assignment is one unit cost.

Answer:

Each inner loop executes n times for each of the n iterations of the outer loop.

Each inner loop iteration performs:

1 (addition) + 1 (assignment) = 2 steps

Total steps: $n \times n \times 2 = 2n^2$

Q33. (Analyze)

Question: You are given two algorithms with the following step-count equations:

• Algorithm A: $5n + 10$

• Algorithm B: $n^2 + 3n + 1$

Which algorithm is more efficient for large n? Justify using RAM model reasoning.

Answer:

As n grows, n^2 dominates linear terms. Therefore:

• Algorithm A is $O(n)$

• Algorithm B is $O(n^2)$

So for large n, Algorithm A is significantly more efficient. For small n, B might be faster due to smaller constants, but asymptotically A is better. RAM model helps identify such trends based on dominant terms, ignoring lower-order constants.

Q34. (Analyze)

Question: Compare the time taken by the two algorithms below for large values of n. Use the RAM model to determine which one will perform better.

```
// Algorithm X

for (int i = 0; i < n; i++) {

total += A[i];

}

// Algorithm Y

for (int i = 0; i < n; i++) {
```

```
for (int j = 0; j < 5; j++) {

total += A[i];

}

}
```

Answer:

Algorithm X performs 1 operation per iteration → total = n operations → T(n) = n

Algorithm Y performs 5 operations per iteration → total = 5n operations → T(n) = 5n

Asymptotically both are O(n), but Algorithm X is faster by a constant factor. RAM model allows comparison of step-counts with constants; here, Algorithm X is better for all n.

Q35. (Evaluate)

Question: Suppose you are given two algorithms:

• Sum1: 2n + 5 steps

• Sum2: 4n + 2 steps

For which values of n (small or large) is Sum1 more efficient? Is there a threshold value of n where their performance changes? Explain.

Answer:

We compare the expressions:

$2n + 5 < 4n + 2$

Solving:

$3 < 2n \rightarrow n > 1.5$

So, for all $n \geq 2$, Sum1 is faster.

Sum2 might appear faster only for very small n (n=1), due to lower constant.

Therefore, Sum1 is asymptotically better, and its advantage grows with large n. RAM model helps derive such break-even points using basic algebra.

Q6. (Apply)

Question: Given the array [4, 3, 1, 5], trace one full pass of the bubble sort algorithm and show the result after that pass.

Answer:

Bubble sort compares adjacent elements and swaps them if they are in the wrong order.

Pass 1 Steps:

Compare 4 and 3 → Swap → [3, 4, 1, 5]

Compare 4 and 1 → Swap → [3, 1, 4, 5]

Compare 4 and 5 → No Swap → [3, 1, 4, 5]

Result after 1 full pass: [3, 1, 4, 5]

Swap Count in this pass: 2

This shows that the largest element (5) has bubbled to the correct position.

Q8. (Evaluate)

Question: You have two arrays:

Array A: 1, 4, 5, 7, 9

Array B: 9, 4, 1, 7, 5

You are to search for the number 5. Which search technique (linear or binary) is more appropriate for each array and why?

Answer:

Array A is sorted in ascending order → Binary search is appropriate

Binary search will use divide-and-conquer, giving O(log n) complexity

Array B is unsorted → Linear search is more appropriate

Binary search cannot be applied unless the array is sorted first

Conclusion:

Use binary search on Array A (efficient due to sorting)

Use linear search on Array B (no sorting, so sequential scan is needed)

Q10. (Apply)

Question: Given the binary search iteration trace:

low = 0, high = 7
mid = 3 → arr[3] = 15
target = 8 → move left
low = 0, high = 2

mid = 1 → arr[1] = 7
target = 8 → move right
low = 2, high = 2
mid = 2 → arr[2] = 8 → found

How many comparisons were made? What was the size of the original array? Explain using the RAM model.

Answer:

Comparisons made: 3

1st: arr[3] = 15 → compared with 8

2nd: arr[1] = 7 → compared with 8

3rd: arr[2] = 8 → compared with 8

Original size of array: 8 (from high = 7 and low = 0)

Using RAM model:

Each iteration has 1 key comparison

In total: 3 iterations → 3 comparisons (unit-cost each)

Time complexity: O(log n)

This aligns with binary search's logarithmic behavior, showing how the RAM model's step count reflects real comparisons.